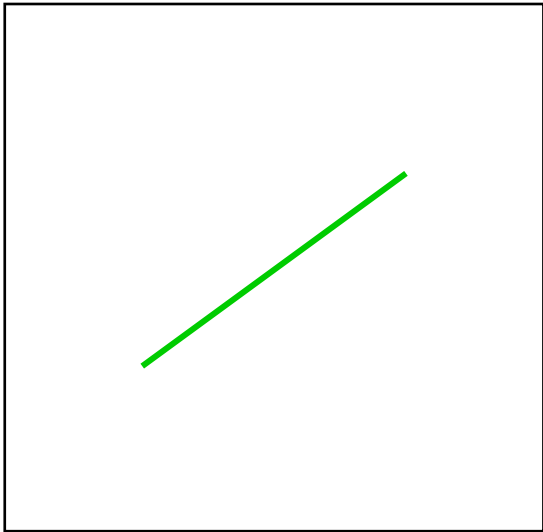


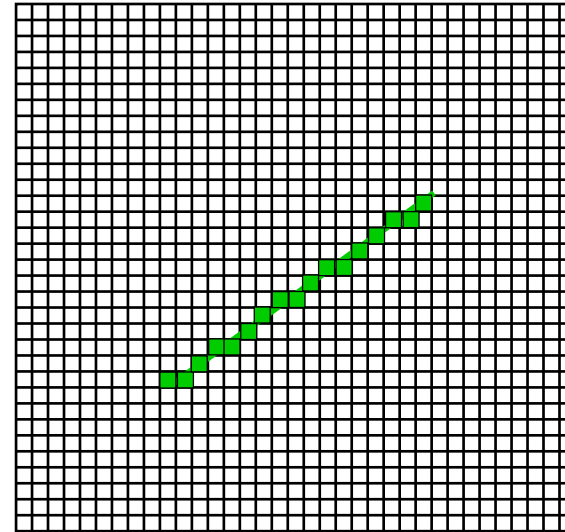
2D Graphics Drawing

Start From Simple

How to draw a line: $y(x) = mx + b$?

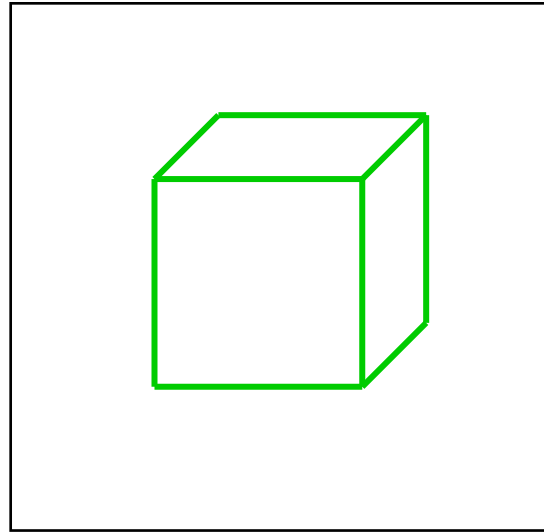


Ideal Picture

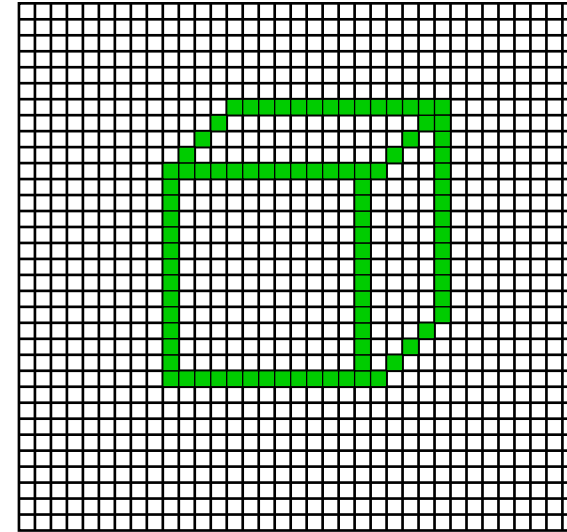
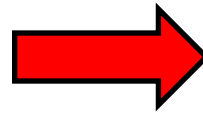


Raster Representation

Scan Conversion, a.k.a. Rasterization



Ideal Picture



Raster Representation

Scan Conversion: Process of converting shapes to raster

Scan Conversion Algorithms

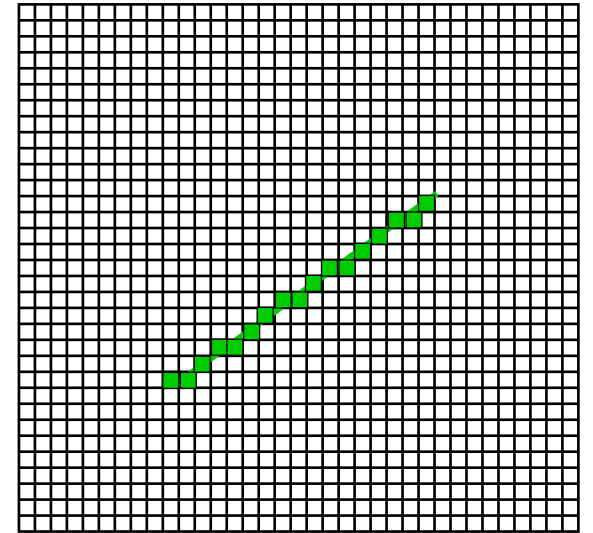
- A **discrete** set of pixels can only **approximate** a continuous geometric object
- This means that scan conversion usually introduces error
- Properties of good scan conversion algorithms:
 - *Accuracy*
 - *Efficiency*
- Challenges
 - Modify *all* the right pixels
 - Touch *only* the right pixels
 - Calculate their values correctly
 - Do it quickly
- *So, start with a correct algorithm and optimize it*

A Really Simple Line Algorithm

- Equation for a line: $y(x) = mx + b$ ($0 \leq x < 1$)
- Step along one pixel at a time in the “fast” direction, here x direction, fill in one pixel per column
- So, just evaluate for each x

```
void line (int x0, int y0, int x1, int y1){  
    float m = (y1 - y0) / (float) (x1 - x0);  
    float b = y0 - m*x0;  
    int x;  
    for(x=x0; x<=x1; x++) { // x is the fast direction  
        float y = m*x + b;  
        draw_pixel(x, Round(y));  
    }  
}
```

- Certainly correct, but slow:
 - integer add, cast to float, floating multiply and add, plus round every step.



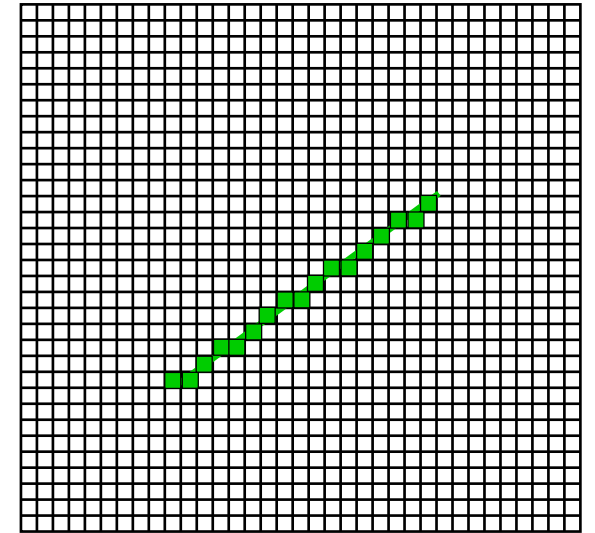
Raster Representation

Lines: DDA (Differential Digital Analyzer) Algorithm

- Optimize the previous to remove multiply from inner loop.
- If we know $y(x)$, we can calculate $y(x+1)$
(note that x is the faster direction, so $m < 1$):

$$y(x+1) = mx + m + b = y(x) + m$$

```
void line (int x0, int y0, int x1, int y1) {  
    float y = y0;  
    float m = (y1 - y0) / (float) (x1 - x0);  
    int x;  
    for (x=x0; x<=x1; x++) {  
        draw_pixel (x, Round(y));  
        y += m;  
    }  
}
```



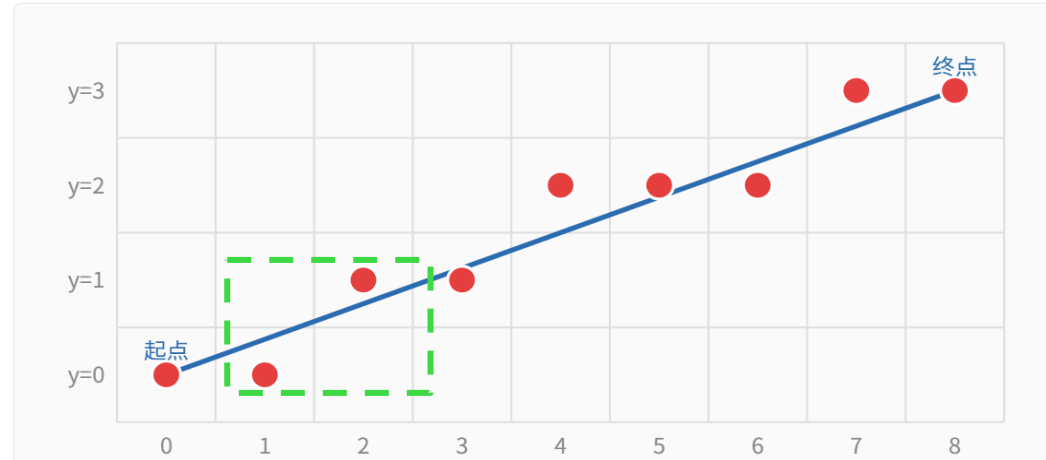
Raster Representation

- This is called *Differential Digital Analyzer* (DDA)
- Problem: Floating-point add and rounds are expensive

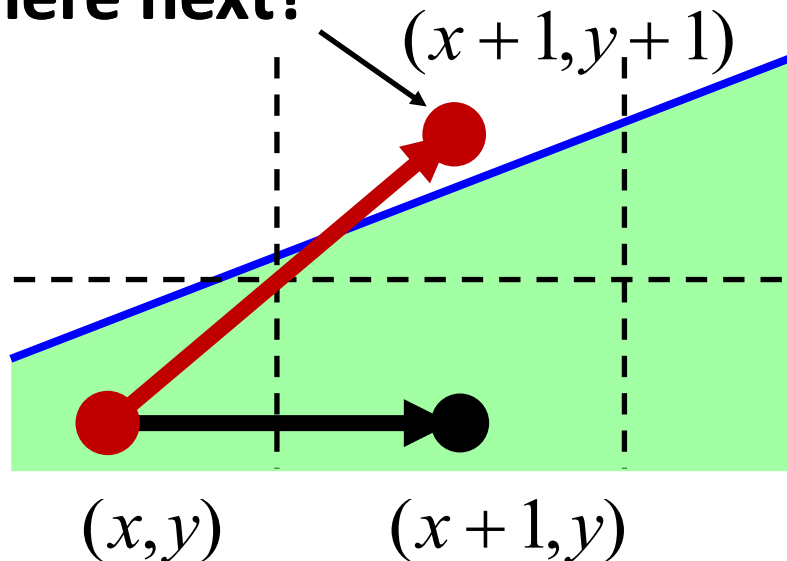
Line Drawing: Which Pixel is Next?

Bresenham 画直线: 从 (0,0) 到 (8,3)

示意 · $dx=8$, $dy=3$, 斜率 $3/8$; 红点为算法选中的像素中心, 蓝线为理论直线



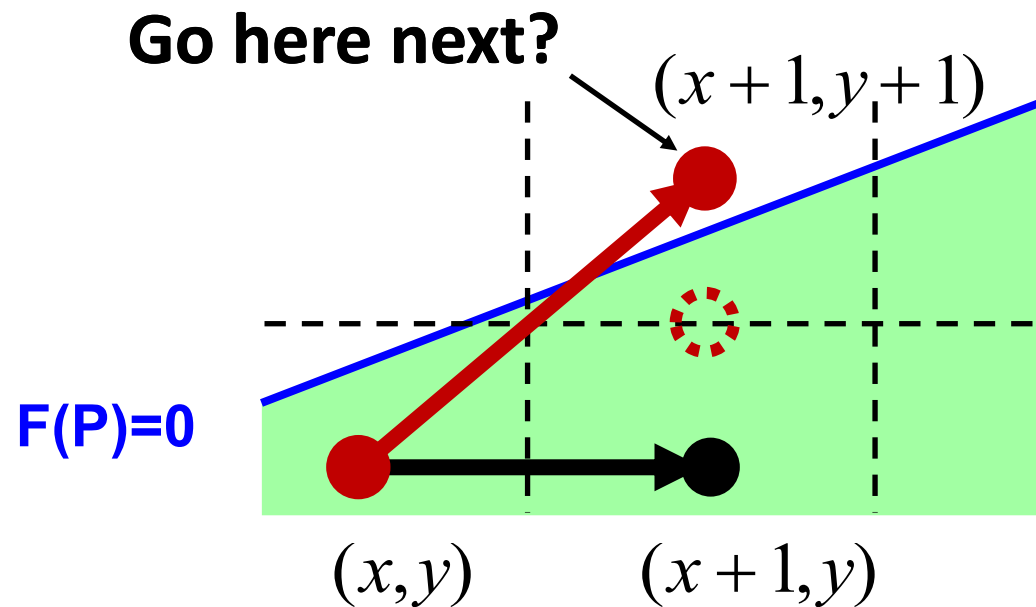
Go here next?



- Assume:
 - $0 < \text{slope} < 1$
 - sorted endpoints, $x_0 < x_1$
- At each step:
 - Current point is (x, y)
 - Next point is pixel $(x+1, ?)$ that's closest to the actual line
 - Do we increment x and y or only x ?
- Use the implicit function to decide!

Use the Implicit Function

- Idea: Test the half-way point $(x+1, y+1/2)$



implicit function $F(P)$,

$F > 0 \rightarrow P$ is above L

$F = 0 \rightarrow P$ is on L

$F < 0 \rightarrow P$ is below L

$F((x+1, y+1/2)) > 0 ?$

yes : increment x and y

no : increment x

Implicit Function for a Line

Line **L** from $[x_0, y_0]$ to $[x_1, y_1]$.

$$\mathbf{P}_0 = [x_0, y_0],$$

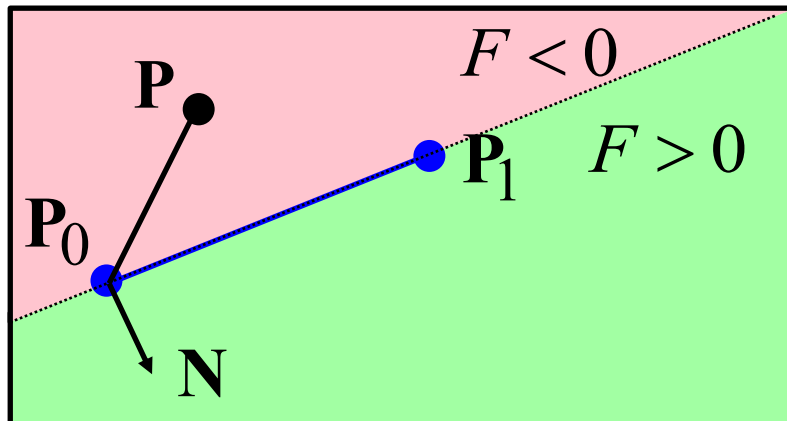
$$\mathbf{P}_1 = [x_1, y_1].$$

$$dx = x_1 - x_0, dy = y_1 - y_0$$

$$\mathbf{N} = [dy, -dx]; \quad // \quad \mathbf{T} = [dx, dy], \quad \mathbf{N} \cdot \mathbf{T} = 0$$

implicit function: $F(\mathbf{P}) = \mathbf{N} \cdot (\mathbf{P} - \mathbf{P}_0)$

$F = 0 \rightarrow \mathbf{P}$ is on **L**



Implicit Function for a Line

Line **L** from $[x_0, y_0]$ to $[x_1, y_1]$.

$$\mathbf{P}_0 = [x_0, y_0],$$

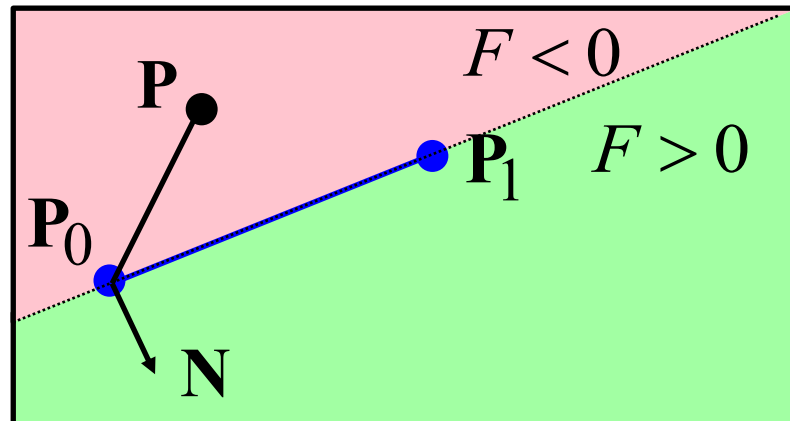
$$\mathbf{P}_1 = [x_1, y_1].$$

$$dx = x_1 - x_0, dy = y_1 - y_0$$

$$\mathbf{N} = [dy, -dx]; \quad // \quad \mathbf{T} = [dx, dy], \quad \mathbf{N} \cdot \mathbf{T} = 0$$

implicit function: $F(\mathbf{P}) = \mathbf{N} \cdot (\mathbf{P} - \mathbf{P}_0)$

$F = 0 \rightarrow \mathbf{P}$ is on **L**



How to remove the *dot product*?

**Trick: Incrementally
Update F**

$$\mathbf{P} = (x, y), \Delta = (1, 1/2)$$

$$F(\mathbf{P}) = \mathbf{N} \cdot (\mathbf{P} - \mathbf{P}_0)$$

$$\begin{aligned} F(\mathbf{P} + \Delta) &= \mathbf{N} \cdot (\mathbf{P} + \Delta - \mathbf{P}_0) \\ &= F(\mathbf{P}) + \mathbf{N} \cdot \Delta \end{aligned}$$

- We only care about the sign of F , so multiply the function by 2 to avoid floating point calculation

Trick: Incrementally Update F

$$F(\mathbf{P}) = 2\mathbf{N} \cdot (\mathbf{P} - \mathbf{P}_0)$$

$$\begin{aligned} F(\mathbf{P} + \Delta) &= 2\mathbf{N} \cdot (\mathbf{P} + \Delta - \mathbf{P}_0) \\ &= F(\mathbf{P}) + 2\mathbf{N} \cdot \Delta \end{aligned}$$

- Computing $F(\mathbf{P})$ requires a dot product:
 - 2 multiplications and 1 add
- But computing $F(\mathbf{P} + \Delta)$ requires only 1 add
 - The $2\mathbf{N} \cdot \Delta$ term is constant - it only needs to be calculated once
- Δ is $[1,0]$, $[1,1]$, or $[1, 1/2]$

Bresenham's Algorithm

$F_0 = F(x_0, y_0) = 0$

$\text{test_}F_0 = F(x_0, y_0) + (1, 1/2) = F_0 + 2N \cdot (1, 1/2)$
 $= F_0 + 2(dy, -dx) \cdot (1, 1/2)$
 $= F_0 + (2dy - dx)$

If $\text{test_}F_0 < 0$

draw($(x_0, y_0) + (1, 0)$)

$F_1 = F(x_0, y_0) + (1, 0) = F_0 + 2(dy, -dx) \cdot (1, 0) = F_0 + 2dy$

$\text{test_}F_1 = \text{test_}F_0 + 2dy$

Else

draw($(x_0, y_0) + (1, 1)$)

$F_1 = F(x_0, y_0) + (1, 1) = F_0 + 2dy - 2dx$

$\text{test_}F_1 = \text{test_}F_0 + 2dy - 2dx$

```
int dx = 2*(x1-x0) ,
Int dy = 2*(y1-y0) ;
int dydx = dy-dx;

F = 0 + dy - dx/2;
for (x=x0 ; x<=x1 ; x++) {
    draw_pixel(x, y);
    if (F<0) F += dy;
    else {y++; F += dydx;}
}
```

Bresenham's Algorithm

This does the right thing (same as DDA) at a **cost of only 2 or 3 integer adds per point.**

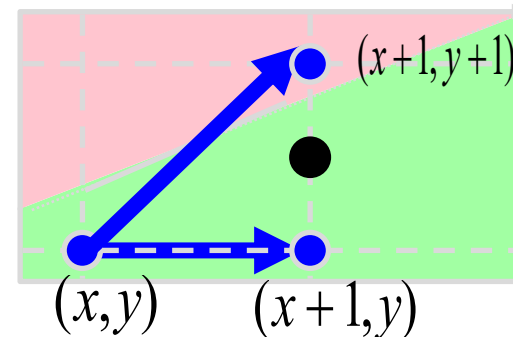
(assumes sorted endpoints, $0 < \text{slope} < 1$)

```
void draw_line(int x0, int y0, int x1, int y1)
{
    int x, y = x0, y0;
    int dx = 2*(x1-x0), dy = 2*(y1-y0);
    int dydx = dy-dx, F = dy-dx/2;

    for (x=x0 ; x<=x1 ; x++) {
        draw_pixel(x, y);
        if (F<0) F += dy;
        else {y++; F += dydx;}
    }
}
```

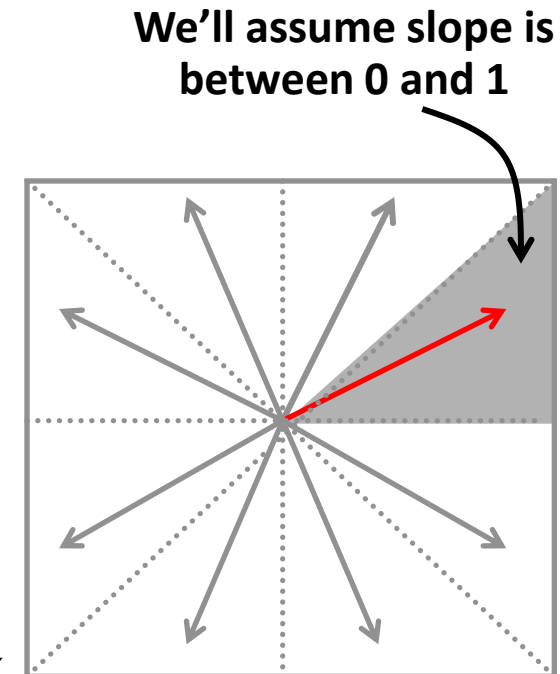
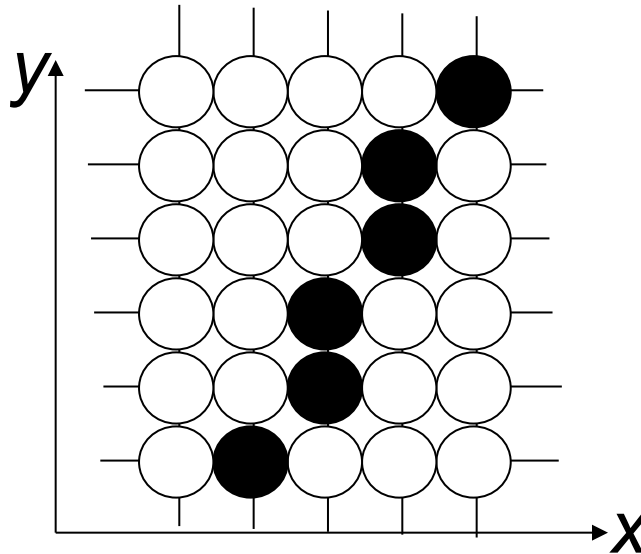
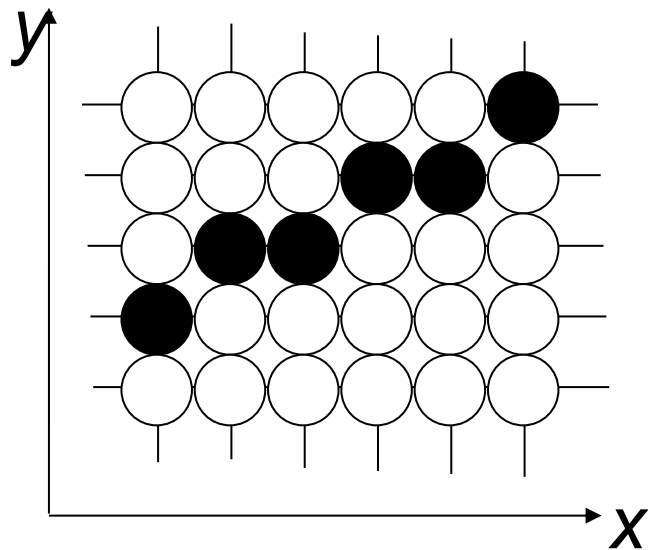
```
int dx = 2*(x1-x0),
int dy = 2*(y1-y0);
int dydx = dy-dx;

F = 0 + dy - dx/2;
for (x=x0 ; x<=x1 ; x++) {
    draw_pixel(x, y);
    if (F<0) F += dy;
    else {y++; F += dydx;}
}
```



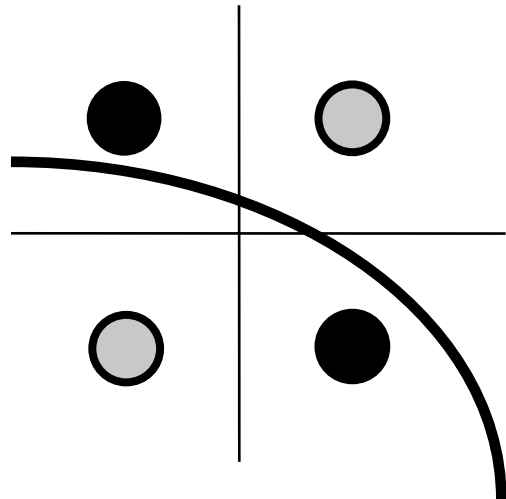
Line Drawing, Cases by Octant

- The algorithms for drawing lines need to step along one pixel at a time in the “fast” direction, which depends on the slope of the line
- We also have to worry about reversed end point order (drawing from large to small X, for example).
- This gives us 8 cases.

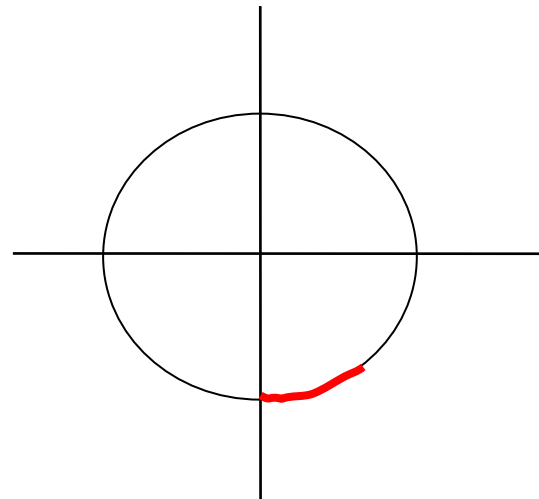


Bresenham Algorithm for Circles

- Same approach as line algorithm
 - use a decision variable formula derived for a circle ($F = x^2 + y^2 - r^2$)
- Eightfold symmetry
 - only compute the points for one octant - use sign flips to give the rest
- Extends to general conics (ellipses...)



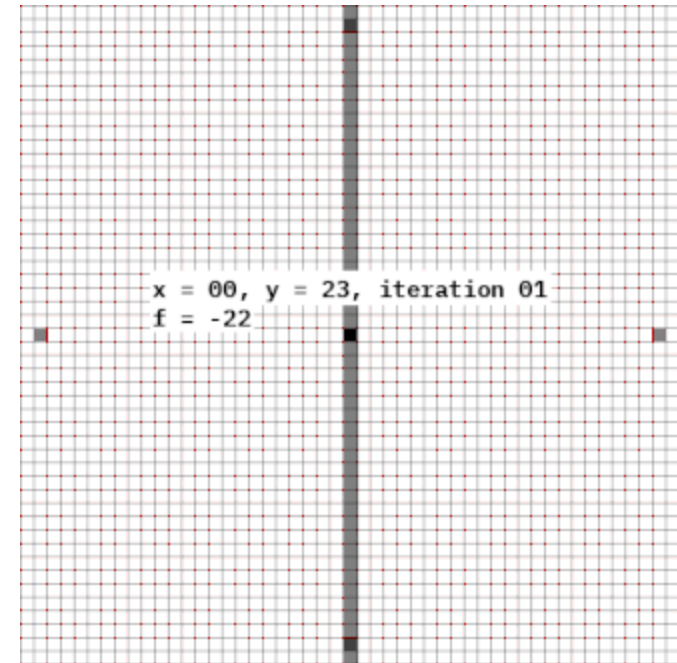
$$(F = x^2 + y^2 - r^2)$$



Bresenham Circle Algorithm

This draws a circle by calculating in one octant and re-using the resulting point 8 times

```
void draw_circle(int radius) {  
    int x = 0, y = radius;  
    int d = 3-2*radius;  
    while (y>x) {  
        if (d<0) /* select East point next */  
            d += 4*x + 6;  
        else { /* select South-East point next */  
            d += 4*(x-y) + 10;  
            y--;  
        }  
        x++;  
        draw_8_pts(x, y); /* draws point in each octant */  
    }  
}
```



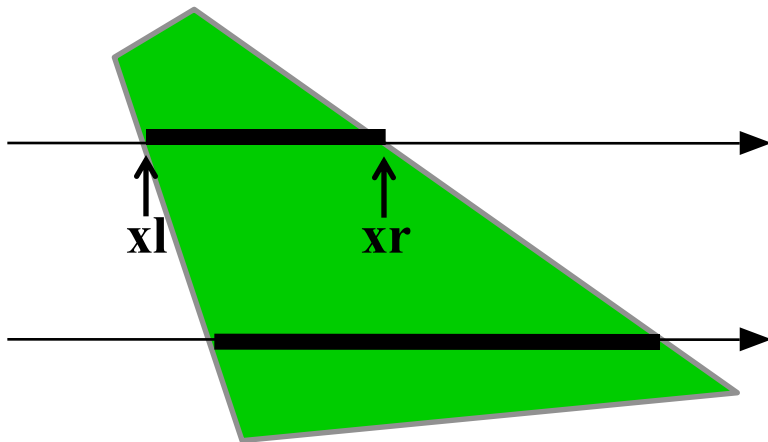
Scan Converting Filled, Convex Polygons

- Find top and bottom vertices
- Make list of edges along left and right sides
- For each scanline from top to bottom
 - There's a single span to fill
 - Find left & right endpoints of span, x_l & x_r , (can use Bresenham's algorithm)

$$x_L += dx_L/dy_L$$

$$x_R += dx_R/dy_R$$

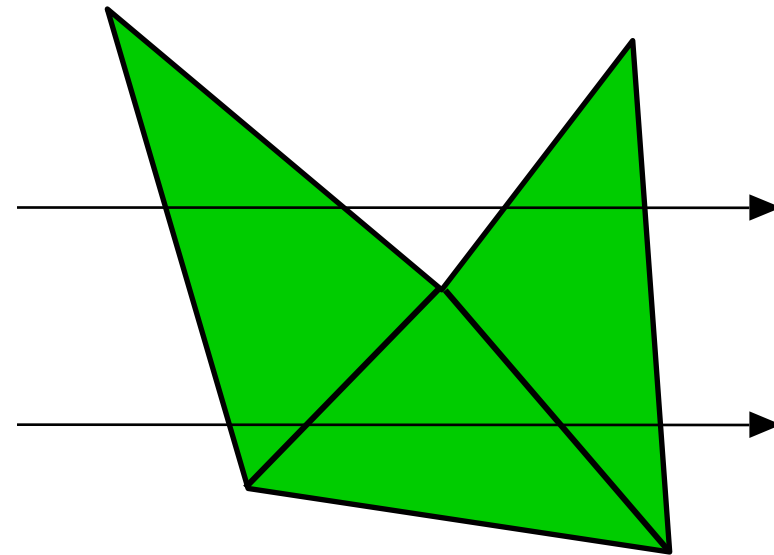
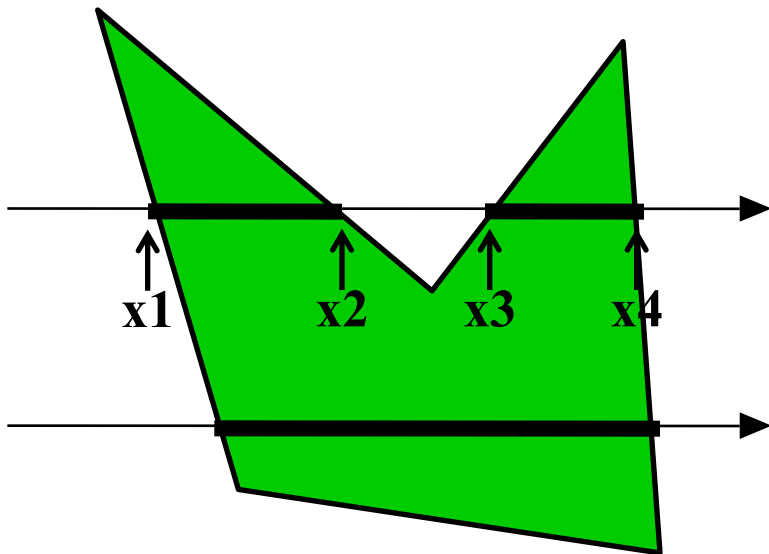
- Fill pixels in-between x_l & x_r
- If you don't do all of the above carefully, cracks or overlaps between abutting polygons result!



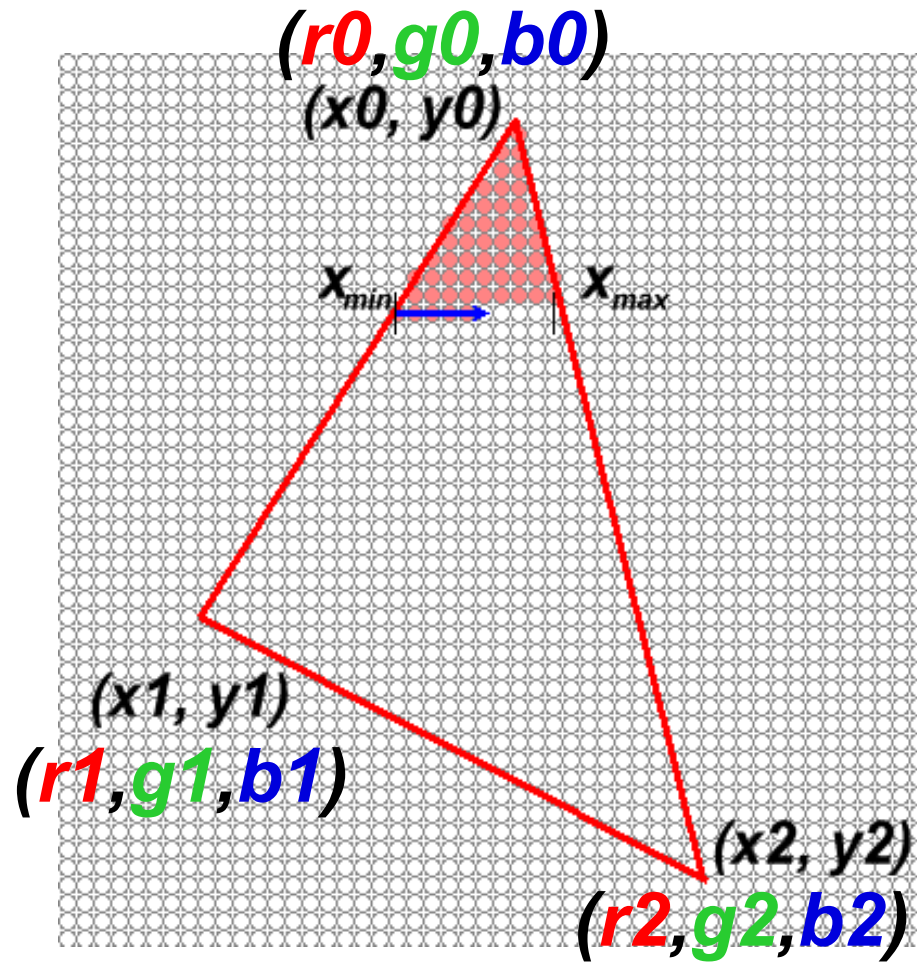
Scan Converting Filled, Concave Polygons

- For each scanline
 - Find **all** the scanline/polygon intersections
 - Sort them left to right
 - Fill the interior *spans* between intersections
 - *Parity Rule* (奇偶规则) : odd ones are interior, even are exterior

Or, triangulation!

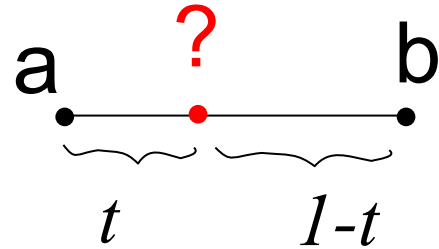


Color Interpolations



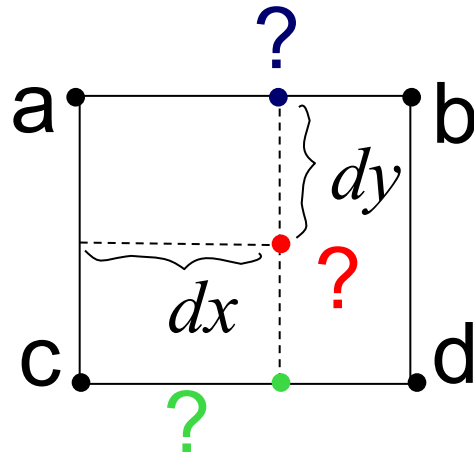
Review on Interpolation

- Linear Interpolation



$$\begin{aligned} ? &= a(1-t) + bt \\ &= a + (b-a)t \end{aligned}$$

- Bilinear Interpolation



$$? = a(1-dx) + bdx$$

$$? = c(1-dx) + ddx$$

$$\begin{aligned} ? &= ?(1-dy) + ?dy = ? + (? - ?)dy \\ &= a(1-dx)(1-dy) + bdx(1-dy) \\ &\quad + c(1-dx)dy + ddx dy \end{aligned}$$

Image Warping

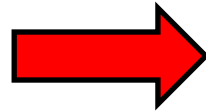


Image Warping

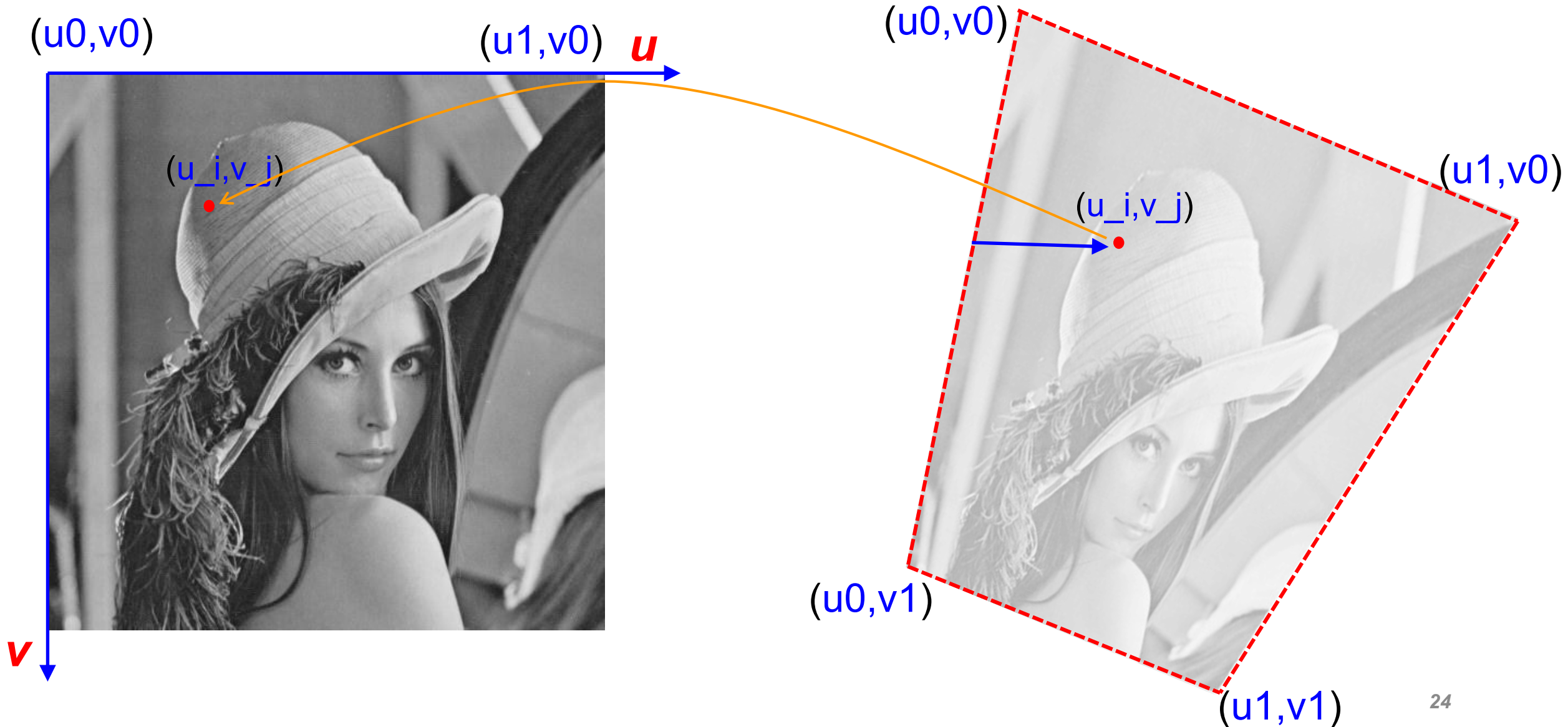
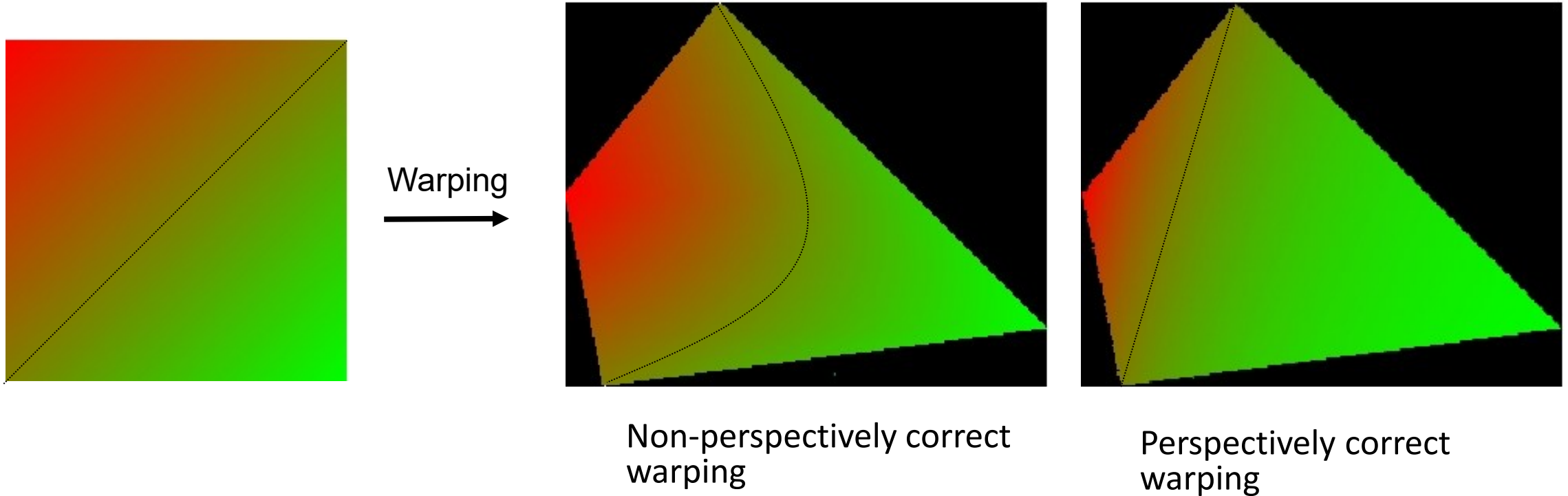


Image warping



More questions on image warping (color interpolation)

- interpolation with perspective warping (later!)



You can write a book about it (image warping)

